

RAM MEMORY ARCHITECTURE DESIGN

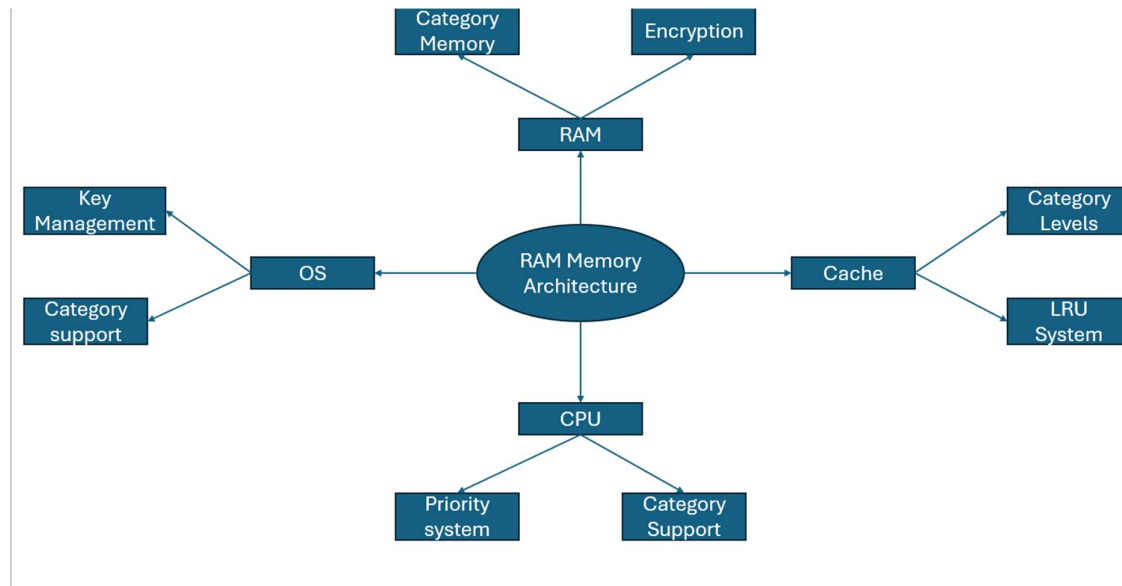
Contents

- Introduction 2**
- RAM Category-Based Memory 3**
- Category-Based Memory Workflow 4**
- RAM Encryption Mode..... 5**
- RAM Memory Architecture Trade-offs 6**

Introduction

This RAM Memory Architecture is designed to improve the performance and security of RAM. This was also designed to address the limitation of flat addressing, which RAM currently uses to store instructions. This RAM memory architecture was created because modern systems and functions such as gaming, AI/ML infrastructure, and server workloads require fast and consistent performance. This RAM Memory Architecture was created with two main features. These features are category-based memory and RAM encryption mode. This design will explain the changes needed and how each feature works.

This Memory Architecture cannot promise perfect performance or security of RAM. However, it can promise to increase performance and reduce workloads compared to the current flat addressing systems. It can also provide protection against cold boot attacks without having to write contents to encrypted disk. This is a design which may be time-consuming for engineers to implement. Some systems may only use the category-based memory or RAM encryption mode while some systems may use both. Make sure to adapt this design based on operating system and CPU architecture if necessary for your own systems.



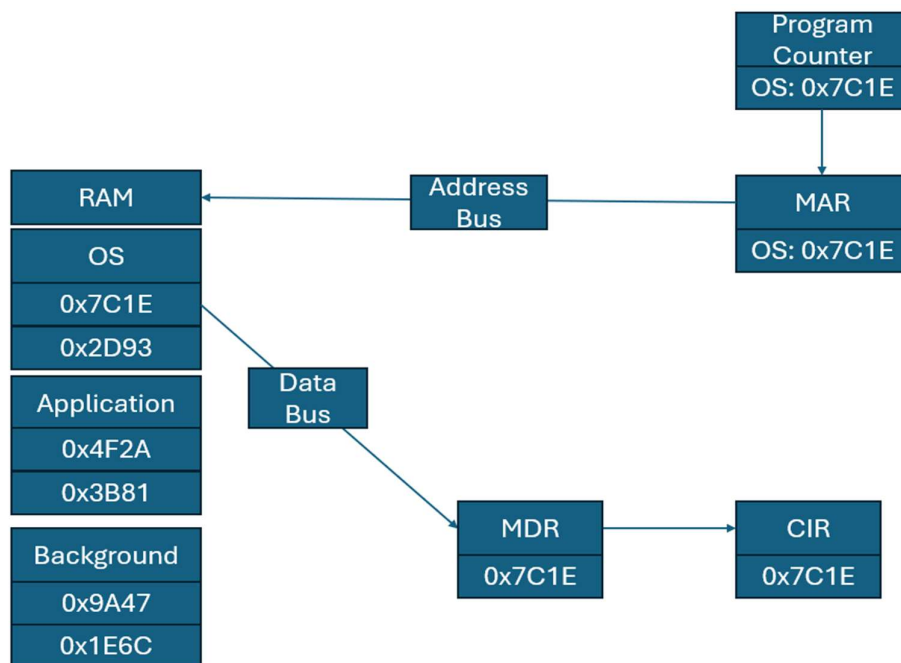
This spider diagram shows the key changes made to RAM, the CPU, Cache and operating system. For those that may not be familiar with computing architecture, LRU stands for least recently used which is used to remove less frequently used data from cache. More detail on changes for cache can be found in the cache design document in this repo.

RAM Category-Based Memory

- **Category-based memory is where instructions for RAM are stored in specific categories instead of just one space.**
- **For example, one category could be applications where instructions used by applications on a device are found.**
- **Category-based memory can be used on both mobile devices and PCs.**
- **There are 3 main categories that should be used by default on most modern systems although more categories could be added for specific use cases.**
- **These categories are operating system, applications, and background.**
- **Operating system category is where operating system processes and instructions are stored.**
- **Applications category is where application and third-party processes and instructions are stored.**
- **Background category is where background services and instructions are stored.**
- **The categories are efficient because the CPU can look for specific instructions within categories leading to faster access times.**
- **RAM Space is allocated by physical RAM modules which you can find out more about in my motherboard design.**
- **However, the operating system still must perform paging, memory allocation, protection, caching, swapping and memory management within categories.**
- **Paging must be configured to move data from RAM to disk while keeping data organised for both disk partitions and RAM categories.**
- **Memory allocation must be configured to allocate RAM for processes based on the RAM available for each category.**
- **Memory Protection must be configured to protect processes from accessing other processes memory in other categories or within its own category and enforcing permissions per category**
- **For example, OS category processes may have different permissions compared to application category processes.**
- **Caching must be configured to store most frequently used files within each category to ensure faster I/O times per category.**
- **Swapping must be configured to only move the data within the category out of RAM space to ensure that not all categories are moved during swapping.**
- **Memory management must be configured to map certain files from partitions and RAM categories.**
- **All these changes would require a complete change in how the operating system manages memory.**
- **However, these changes would make RAM usage more efficient and decrease look-up times overall increasing performance**
- **It is important to remember that these changes may differ on mobile devices due to ARM architecture.**

Category-Based Memory Workflow

- This workflow is designed to show engineers or users how the CPU process would work.
- First, the program counter points to the next category and address in memory.
- For example, this could look like OS: 0x7C1E within the program counter.
- Second, the category and address are then temporarily stored in the MAR.
- MAR stands for memory address register for those who may not know.
- Third, the address bus heads to the OS category in RAM.
- The address bus then navigates the OS category to find the specific address.
- Fourth, once the address is found, the data bus transports the instruction.
- This instruction is stored temporarily in the memory data register, also known as MDR.
- Fifth, the instruction is then sent to the current instruction register also known as CIR.
- In the CIR, the instruction is decoded into operand and opcode.
- Opcode tells the CPU what to do with data, while operand tells the CPU what data to perform that action on.
- Sixth, the CPU then executes the instruction using other components such as the ALU where needed.
- Finally, the program counter increments by one and points to the next category and address in memory.
- For example, this could look like App: 0x4F2A



The spider diagram above shows what RAM Architecture looks like when using category-based memory. This diagram does not include cache due to the focus on RAM.

RAM Encryption Mode

- RAM encryption mode is designed to make sure the contents of RAM are protected due to the sensitivity of data stored there.
- This RAM encryption is designed for enterprise environments and high-risk users who may face nation-state actors.
- This RAM encryption mode works differently from traditional hibernation mode in terms of efficiency and security.
- Traditional hibernation mode writes the contents of RAM to the encrypted disk which is slower and is dependent on disk encryption and security.
- RAM encryption mode encrypts RAM in place during hibernation mode or a certain state. After the user authenticates, the RAM contents are decrypted.
- The encryption and decryption processes is managed by the operating system as in modern computing systems that is where key management is handled.
- The encryption key should be stored in TPM (Trusted Platform Module) which is important because the encryption key can't be stored in system RAM.
- The operating system can give users control over what authentication method is used and the encryption algorithm used.
- For example, users may have the option to tie the key to their account password or a separate 8-digit PIN.
- For encryption algorithms, users could use AES, Serpent or Twofish to have control over how their data is protected.
- For recovery, a one-time key is shown to the user which is stored securely by the OS on the device via TPM or an encrypted partition.
- This means that the user is responsible on how they store and secure the key as if they lose the key then the RAM session is lost.
- There should be no physical reset switch to prevent an attacker with physical access from using the recovery mechanism to compromise RAM.
- However, it is important to remember that you won't be locked out of your device since RAM is volatile meaning all data is lost when power is lost.
- There are 3 important notes about this RAM encryption mode.
- The first note is that it may not be compatible with mobile devices due to ARM architecture however, this security feature may not be needed for mobile devices.
- The second note is that this RAM encryption mode will not be effective without TPM as it is the most secure key management method
- This is because if the key is stored within TPM it is never exposed and it also prevents the circular problem of using RAM to encrypt RAM.
- The third and final note is that traditional hibernation can still be used on a device if TPM is not present.
- However, this means that the reliance of security is on the disk's encryption and authentication.
- So each option has its trade-offs so make sure to choose the approach wisely as they can make a difference in high-risk environments.

RAM Memory Architecture Trade-offs

- This RAM memory architecture has trade-offs just like any other system I have built.
- There are 5 main trade-offs that I have identified.
- Engineers may face more trade-offs during implementation as this is only a design.
- The first trade-off is compatibility.
- This is because the encryption mode may not be compatible with mobile device due to the ARM architecture.
- The encryption mode also requires the device to have an TPM which not all devices support.
- This trade-off can be managed by targeting PC/laptops, using other secure storage methods on the OS or on the cloud and focusing on encryption mode for the targeted audience.
- The second trade-off is performance.
- This is because the encryption mode can cause latency due to encrypting and decrypting the contents of RAM especially if the RAM size is large.
- This trade-off can be managed by keeping the encryption mode strictly to hibernation mode and using pre-existing operating system authentication.
- The third trade-off is lockout risk.
- This is because the encryption mode may lockout the user if they forget their password.
- This trade-off can be managed through the recovery key (No data lost) or turning off the device which means all data in RAM is lost.
- The fourth trade-off is heat.
- This is because since the CPU can fetch instructions more quickly it can also execute more instructions per cycle.
- This can cause the CPU to heat up a lot more than flat addressing.
- This trade-off can be managed by cooling systems and clock speed within the CPU.
- The fifth trade-off is legacy support.
- This is because this RAM memory architecture requires major changes for the CPU, updates to the operating system and changes to RAM.
- This can take years, and most users will not be able to adapt from current hardware easily.
- This trade-off can be managed through new operating system memory controller, new generation of CPU and a new RAM module.
- This legacy support trade-off will be the most difficult and expensive to migrate.
- There may be some more migrations that can be used for these trade-offs, I have not considered.
- Make sure to adapt migrations based on resources and budget.